SPIN - Seminar

–

Formal Methods for AI-Based Systems Engineering (FMAISE)

Robert Li & William Tossici (1983696, 1844504)

Academic Year 2025/2026

SAPIENZA
UNIVERSITÀ DI ROMA

Table of Contents

Promela in a nutshell

1 Recap

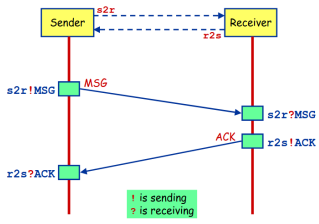


Figure: Some communication protocol

```
mtype = { MSG, ACK };
chan s2r = [0] of { mtype };
chan r2s = [0] of { mtype };

proctype Sender() {
    s2r ! MSG; /* Sender sends MSG */
    r2s ? ACK; /* Sender waits to receive ACK */
    printf("Sender: Successfully sent MSG and
           received ACK.\n");
}

proctype Receiver() {
    s2r ? MSG; /* Receiver waits to receive MSG */
    r2s ! ACK; /* eceiver sends ACK back */
    printf("Receiver: Successfully received MSG
           and sent ACK.\n");
}

init {
    atomic {
        run Sender();
        run Receiver();
    }
}
```

Promela in a nutshell: Execution Model

1 Recap

- **Asynchronous Interleaving:** Processes execute concurrently. SPIN explores *all possible* execution orderings (interleavings) to find hidden edge cases.
- **Non-Determinism:**
  - Managed via `if :: ... fi` and `do :: ... od` structures.
  - If multiple conditions (guards) are true, SPIN chooses one non-deterministically.
- **Controlling Concurrency:**
  - `atomic {...}`: Executes a sequence of statements in a single, uninterruptible step (crucial to avoid unwanted interleaving).
- **Correctness Claims:** Using `assert(...)` statements or LTL formulas to define safety conditions that *must never* be violated.

Table of Contents

2 Doubts

How “;” works

2 Doubts

The key point is that the `atomic` statement in the `init` block only ensures that the three processes are **spawned** without interruption.

However, once started, the execution of `P(0)` and `P(1)` is *interleaved*.

Since the check `flag != 1` and the assignment `flag = 1` are not a single atomic operation, both processes can enter the critical section simultaneously.

```
bit flag;     /* signals entry/exit */
byte mutex;   /* procs in critical section */

proctype P(bit i) {
    flag != 1; /* wait until other leaves */
    flag = 1;  /* PROBLEM: race condition */
    mutex++;
    printf("MSC: P(%d) entered.\n", i);
    mutex--;
    flag = 0;
}

proctype monitor() {
    assert(mutex != 2);
}

init {
    atomic {
        run P(0);
        run P(1);
        run monitor();
    }
}
```

Table of Contents

Mutual exclusion problem (again)

3 Message Sequence Chart

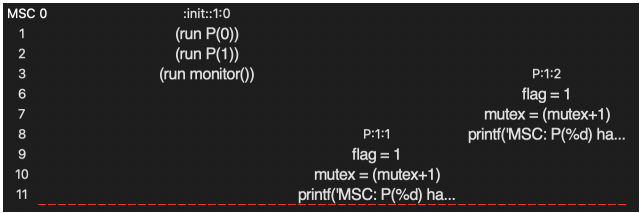


Figure: MSC

```
bit flag;     /* signals entry/exit */
byte mutex;   /* procs in critical section */

proctype P(bit i) {
    flag != 1; /* wait until other leaves */
    flag = 1;  /* PROBLEM: race condition */
    mutex++;
    printf("MSC: P(%d) entered.\n", i);
    mutex--;
    flag = 0;
}

proctype monitor() {
    assert(mutex != 2);
}

init {
    atomic {
        run P(0);
        run P(1);
        run monitor();
    }
}
```

Table of Contents

4 Promela peculiarity

The Goal: Global Invariants

4 Promela peculiarity

- **Objective:** Verify a global property P holds in *every* state.
- **LTL Formula:** `[]` `P` (Always P).
- **Challenge:** How to implement this efficiently in Promela without state space explosion?

Variant 1+2: The Monitor Process

4 Promela peculiarity

Approach: Add a parallel process to check the assertion.

```
1 active proctype monitor() {
2     assert(P);
3 }
```

- **Drawback:** The `assert` is executable in *every* system state.
- Causes massive interleaving.
- Huge increase in the state space.

Variant 3: The Guarded Monitor

4 Promela peculiarity

Approach: Use `atomic` blocking to pause the monitor.

```
active proctype monitor() {
    atomic {
        !P -> assert(P);
    }
}
```

- **Advantage:** Process only wakes up when P is *false*.
- If P is true, `!P` blocks execution.
- Highly efficient, minimal extra states.

Variant 4: The Looping Monitor

4 Promela peculiarity

Approach: Keep the monitor alive in an infinite loop.

```
active proctype monitor() {
    do
    :: assert(P)
    od
}
```

- **Observation:** Seems wasteful (infinite loop).
- **Reality:** Better than V1/V2 because the process never reaches an `-end-` state.
- Reduces the total number of unique states for SPIN to track.

Variant 5: Manual Never Claim

4 Promela peculiarity

Approach: Use SPIN's native `never` construct manually.

```
never {
    do
    :: assert(P)
    od
}
```

- **Warning:** SPIN complains about "Partial Order (p.o.) reduction".
- Manual never claims might not be "stutter-closed".
- Optimization algorithms might produce invalid results.

Variant 6: LTL Property (The Best Way)

4 Promela peculiarity

Approach: Let SPIN do the math. Define the LTL formula.

- **Input:** `[] P`
- **Action:** SPIN negates it (`! []P`) and generates the optimal Automaton.

```
1 never { /* !([]P) */
2 T0_init:
3     if
4     :: (!P) -> goto accept_all
5     :: (1) -> goto T0_init
6     fi;
7 accept_all:
8     skip
9 }
```

Variant 7: The 'unless' Hack

4 Promela peculiarity

Approach: Force an interrupt from inside the process.

```
1 { body } unless {
2     atomic { !P -> assert(P); }
3 }
```

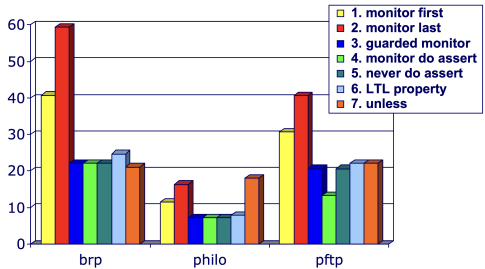
- **Pros:** Saves memory (no extra process), access to local variables.
- **Cons:** Breaks `atomic` blocks inside `body`.
- **Cons:** Disables Partial Order Reduction.
- Not recommended for modern verification.

Invariance experiments

4 Promela peculiarity

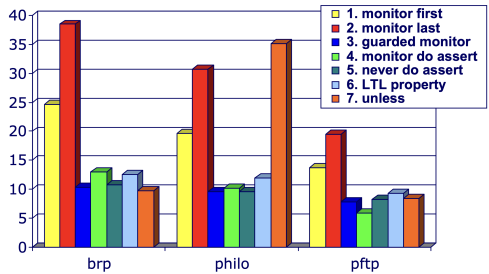
-DNOREDUCE memory (Mb)

NO partial order reduction



-DNOREDUCE time (sec)

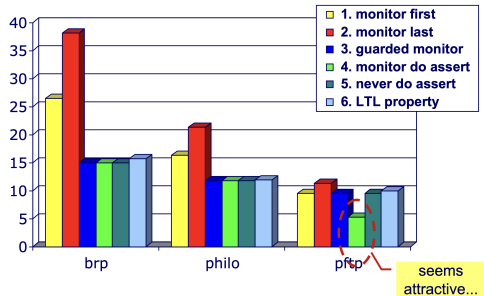
NO partial order reduction



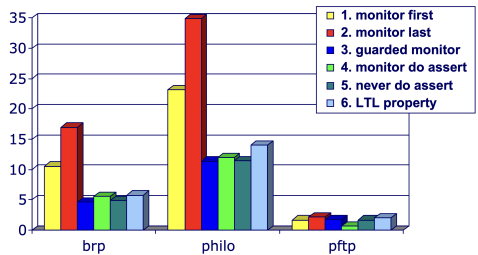
Invariance experiments

4 Promela peculiarity

default settings
memory (Mb)



default settings
time (sec)



Optimizing the Model: `xr`, `xs`, and `_`

4 Promela peculiarity

Goal: Prevent state-space explosion by explicitly telling SPIN how variables are used.

1. **Exclusive Channel Assertions (`xr`, `xs`)**
   - `xr` **and** `xs`**:** Assert that only *one* specific process reads/writes from/to a channel.
   - **Advantage:** Allows SPIN to use aggressive *Partial Order Reduction*.

```
1 chan q = [3] of { byte, byte };
2 xr q; /* Promises SPIN: I am the sole reader */
```

Optimizing the Model: `xr`, `xs`, and `_`

4 Promela peculiarity

Goal: Prevent state-space explosion by explicitly telling SPIN how variables are used.

1. Exclusive Channel Assertions (`xr`, `xs`)

- `xr` **and** `xs`**:** Assert that only *one* specific process reads/writes from/to a channel.
- **Advantage:** Allows SPIN to use aggressive *Partial Order Reduction*.

```
1 chan q = [3] of { byte, byte };
2 xr q; /* Promises SPIN: I am the sole reader */
```

2. The Anonymous Variable (`_`)

- A write-only, ”throwaway” or scratchpad variable.
- **Advantage:** It is **never stored in the state vector**.

```
1 q ? target, _ ; /* Read the first byte into 'target', discard the second */
```

Process Management: _pid, _nr_pr, and Lifecycle

Built-in System Variables

- `_pid`: A local, read-only variable holding the unique ID of the current process. Ranges from 0 to 254. (Process 0 is usually `init`).
- `_nr_pr`: A global, read-only variable showing the total number of currently active processes. **The absolute maximum is 255.**

Process Management: `_pid`, `_nr_pr`, and Lifecycle

4 Promela peculiarity

Built-in System Variables

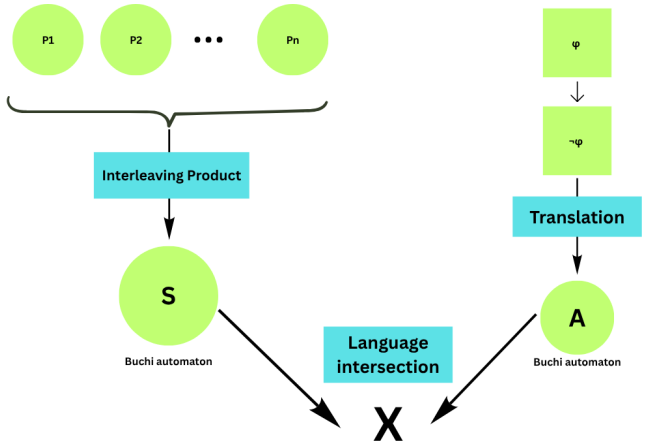
- `_pid`: A local, read-only variable holding the unique ID of the current process. Ranges from 0 to 254. (Process 0 is usually `init`).
- `_nr_pr`: A global, read-only variable showing the total number of currently active processes. **The absolute maximum is 255.**

The Process Lifecycle

- **Birth:** Created statically via `active` or dynamically via `run`. Any process can spawn new processes!
- **Execution:** Transitions execute. A process **cannot** be externally killed.
- **Death:** A process terminates and frees its memory **only** when it successfully reaches its closing brace (}).
  - **Optimization Danger:** Processes strictly die in the **reverse order they were created** (LIFO stack). So the process truly disappears if all child processes have terminated first.

Under the Hood: Promela to Finite State Automaton

4 Promela peculiarity



Under the Hood: Promela to Finite State Automaton

4 Promela peculiarity

SPIN translates every `proctype` into an FSA (control-flow automaton) defined by the 5-tuple: (S, s_0, L, T, F).

- S **(States):** The set of states of this automaton S corresponds to the possible points of control within the proctype
- s_0 **(Initial State (one!)**
- T **(Transitions):** Transition relation `T` defines the flow of control.
- L **(Labels):** The transition label set `L` links each transition in T with a specific basic statement that defines the executability and the effect of that transition
- F **(Final States):** The set of final states `F`, finally, is defined with the help of PROMELA end-state, accept-state, and progress-state labels.

The CFA: Separating Structure from Semantics

4 Promela peculiarity

"Elements such as `if`, `goto`, the statement separators `;` and `->`, and similarly also `do`, `break`, `unless`, `atomic`, and `d_step`, cannot appear as labels on transitions: only the six basic types of statements in PROMELA can appear in set L."

The Code Structure (Syntax)

- **Elements:** `if`, `do`, `goto`, `->`, `;`, `atomic`
- **Role:** Syntactic "Wiring"
- *Note:* These build the shape of the graph, but disappear in the final CFA.

The Program Semantics (Set L)

- **Elements:** The 6 Basic Statements (Assignments, Assertions, Expressions, Sends, Receives, Prints)
- **Role:** Executable Transitions
- *Note:* These are the ONLY elements that sit on the arrows of the CFA.

The CFA: Separating Structure from Semantics

4 Promela peculiarity

"Elements such as `if`, `goto`, the statement separators `;` and `->`, and similarly also `do`, `break`, `unless`, `atomic`, and `d_step`, cannot appear as labels on transitions: only the six basic types of statements in PROMELA can appear in set L."

The Code Structure (Syntax)

- **Elements:** `if`, `do`, `goto`, `->`, `;`, `atomic`
- **Role:** Syntactic "Wiring"
- *Note:* These build the shape of the graph, but disappear in the final CFA.

The Program Semantics (Set L)

- **Elements:** The 6 Basic Statements (Assignments, Assertions, Expressions, Sends, Receives, Prints)
- **Role:** Executable Transitions
- *Note:* These are the ONLY elements that sit on the arrows of the CFA.

The Control Flow Automaton strips away how the code is organized to reveal only what the code actually **does**.

Under the Hood: Promela to Finite State Automaton

4 Promela peculiarity

Let’s see an example of a Promela model and its automaton. Notice how not all arrows are present! Notice the difference with the Program Graph!

Undermodeling in SPIN: The Atomic Increment

4 Promela peculiarity

The Concept: Promela executes assignments like `i++` or variable assignments like `x=y-1` as a single, indivisible step, which is not always the case for other programming languages. This can, in turn, lead to undermodelling where we miss real reachable states in the real system. Let's see an example of it in Promela!

Table of Contents

5 Example from literature: TSP

Why TSP?

5 Example from literature: TSP

- state-explosion problem
- optimization problem
- Branch-and-Bound
- NP-Hard

Traveling Salesman Problem

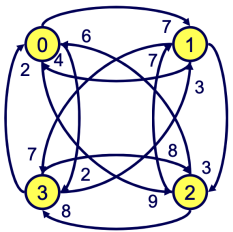
5 Example from literature: TSP

Problem:

- n cities
- cost c_{ij} between city i and j
- non-Euclidean: $c_{ij} \neq c_{ji}$

Goal:

- connect the cities with the **shortest** closed tour, passing each exactly once.



	0	1	2	3
0	-	7	9	2
1	4	-	3	7
2	6	7	-	8
3	2	3	8	-

Figure: Example $n = 4$

Traveling Salesman Problem: algorithm

5 Example from literature: TSP

Approach: reformulate the optimisation problem in terms of reachability, i.e. as (im)possibility to reach a state that improves on a given optimality criterion.

- **Input:** `Promela model M with cost added to the states.`
- **Output:** `the optimal solution min for the optimisation problem of M`

```
1 min ← (worst case) maximum cost
2 do
3     use Spin to check M ⊨ ◇ (cost ≥ min)
4     if (error found)
5     then min ← cost
6 while (error found)
```

NOTE: we care only about non-Euclidean TSP